SECURE DIGITAL BANKING PLATFORM

Milestone 4 - Project Report

Document OCR • Liveness Detection • Face Match Verification

1. Introduction

Secure is a digital banking platform being built incrementally across milestones. Milestone 4 adds an AI-integrated KYC (Know Your Customer) module that automates identity verification for new and existing customers through three AI-assisted checks: document OCR extraction, camera-based liveness detection, and face-match verification against the customer's registered photo.

Team C delivered this milestone as three coordinated layers — an Angular front end, a Spring Boot REST backend, a Python/FastAPI face-recognition microservice, and a MySQL database — with the new KYC tables linked to the existing customer schema from earlier milestones via customer_id.

2. Objectives

- Automate identity document verification using OCR, reducing manual data entry during customer onboarding.
- Confirm that the person completing KYC is a real, live individual (anti-spoofing) via camera-based liveness checks.
- Verify that the live selfie matches the photo on the submitted identity document using AI face comparison.
- Persist every AI decision (scores, confidence, pass/fail) against the customer record for audit and compliance.
- Present a clear, step-by-step verification journey in the banking UI, ending in a consolidated verification summary.

3. System Architecture

The module follows a four-layer architecture. The Angular UI drives the customer through Document Upload → Document OCR → Liveness Check → Selfie Capture → Face Match → Verification Result. The Spring Boot backend exposes REST endpoints for OCR and face-match orchestration and persists results to MySQL/PostgreSQL. A dedicated Python FastAPI microservice performs the actual face-recognition computation using DeepFace, called internally by the backend.

Layer	Technology	Responsibility
Presentation	Angular 19 app (Frontend-M4)	Document upload, OCR review, liveness check, selfie capture, face-match capture
API / Orchestration	Spring Boot service (Backend-M4)	Document OCR endpoint, face-match orchestration, persistence via Spring Data JPA
AI Computation	FastAPI micro-service (face-recognition-service)	DeepFace-based face verification, called internally by the backend
Data	MySQL / digital_banking schema	document_ocr, liveness_detection, face_match tables keyed on customer_id

Table 1 — Layered architecture of the Milestone 4 KYC module

3.1 Request Flow

- Customer uploads an identity document in the Angular UI → OCR extracts name, ID number, DOB, gender and address (client-side via Tesseract.js, or via the backend /api/document-ocr endpoint) → result stored in document_ocr.
- Customer completes a webcam-based liveness challenge (e.g., blink / turn head) in the browser to confirm a real, live person is present → outcome recorded in liveness_detection.
- Once liveness is confirmed, a still selfie photo is captured from the camera feed and held for the face-match step.
- The captured selfie and the customer's registered document photo are sent to the Spring Boot /api/face/verify endpoint → forwarded to the FastAPI /face-match endpoint → DeepFace (VGG-Face model) returns a match verdict and confidence → result stored in face_match.
- The Verification Summary page aggregates all three results into one overall KYC decision.

4. Technology Stack

Layer	Technology	Purpose
Frontend	Angular 19, Angular Material, Tesseract.js	Customer-facing KYC workflow UI and client-side OCR
Backend API	Spring Boot 3.5 (Java 17), Spring Data JPA, RestTemplate	REST orchestration, persistence, service-to-service calls
AI Face Service	Python, FastAPI, DeepFace (VGG-Face), OpenCV	Face verification / similarity scoring microservice
Database	MySQL (phpMyAdmin/XAMPP) — PostgreSQL in backend config	Stores OCR, liveness and face-match results linked to customer_id

5. Module 1 — Document OCR

Customers upload an identity document (Aadhaar, PAN, Passport, Driving Licence or Voter ID). Text is extracted either through the backend REST endpoint or, in the current build, directly in the browser using the Tesseract.js WebAssembly OCR engine, with canvas-based grayscale and contrast preprocessing to improve accuracy.

5.1 Extraction Logic

- Validates file type (JPG/JPEG/PNG) and a 5 MB size limit before processing; PDFs are rejected with a clear conversion message.
- Runs pattern matching per document type — e.g. a 10-character PAN regex, a 12-digit grouped Aadhaar regex, a passport letter+7-digit regex, and a state-code driving-licence regex.
- Parses name, date of birth, gender and address using line-level heuristics, and marks the record VERIFIED once a name and ID number are both found, otherwise NEEDS_REVIEW.

5.2 Backend Endpoints

Method	Endpoint	Description
POST	/api/document-ocr	Submit a document for OCR processing and persist the result

Method	Endpoint	Description
GET	/api/document-ocr/{id}	Retrieve a previously processed OCR record by ID

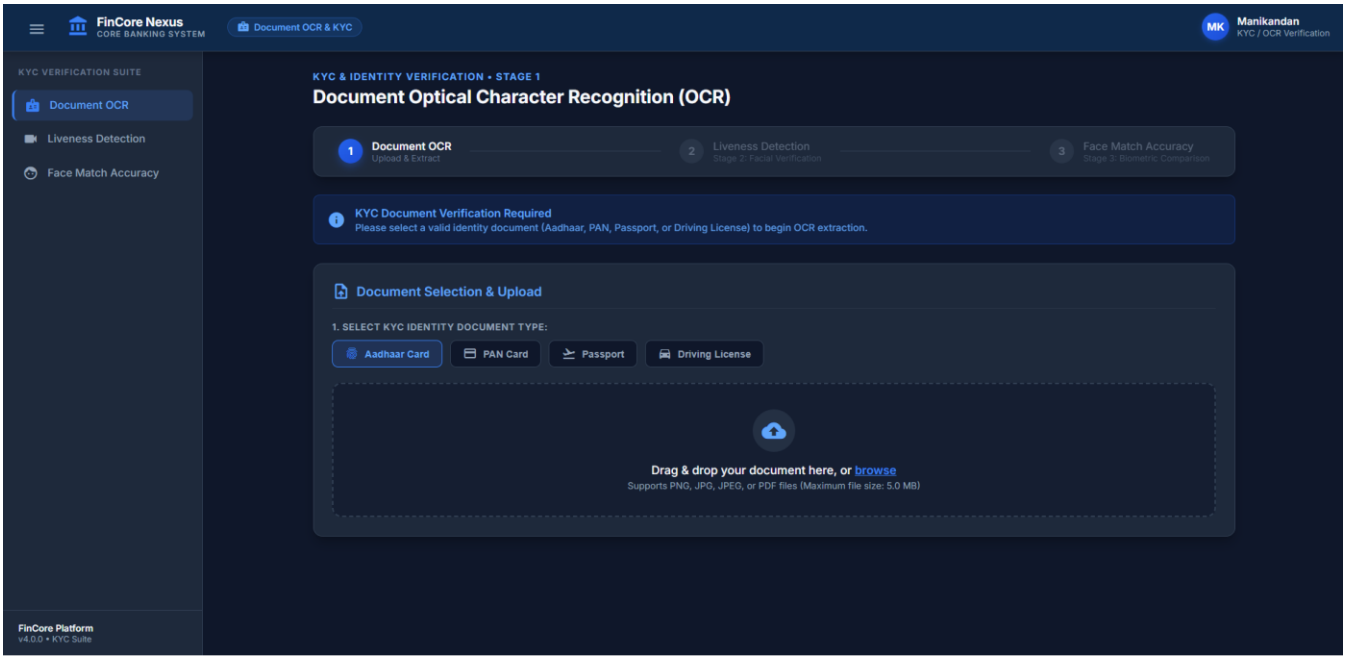

Figure 1 — Document OCR

6. Module 2 — Liveness Detection

Before face matching, the platform confirms the customer is a live person in front of the camera — guarding against photo or video spoofing. The Angular liveness component requests webcam access, walks the customer through a short sequence of challenge steps, and records a liveness score against a configurable pass threshold.

- Camera stream captured via the browser MediaDevices API (getUserMedia).
- Each completed challenge step contributes to an overall liveness_score, compared against a default liveness_threshold of 80%.
- Outcome (Passed / Failed / Pending) and the checked_at timestamp are persisted in the liveness_detection table.

6.1 Backend Endpoints

Method	Endpoint	Description
POST	/api/liveness/verify	Submit the completed challenge result and captured frame for liveness scoring, and persist the outcome
GET	/api/liveness/{id}	Retrieve a previously recorded liveness result by ID
GET	/api/liveness/customer/{customerId}	Fetch the latest liveness verification status for a given customer

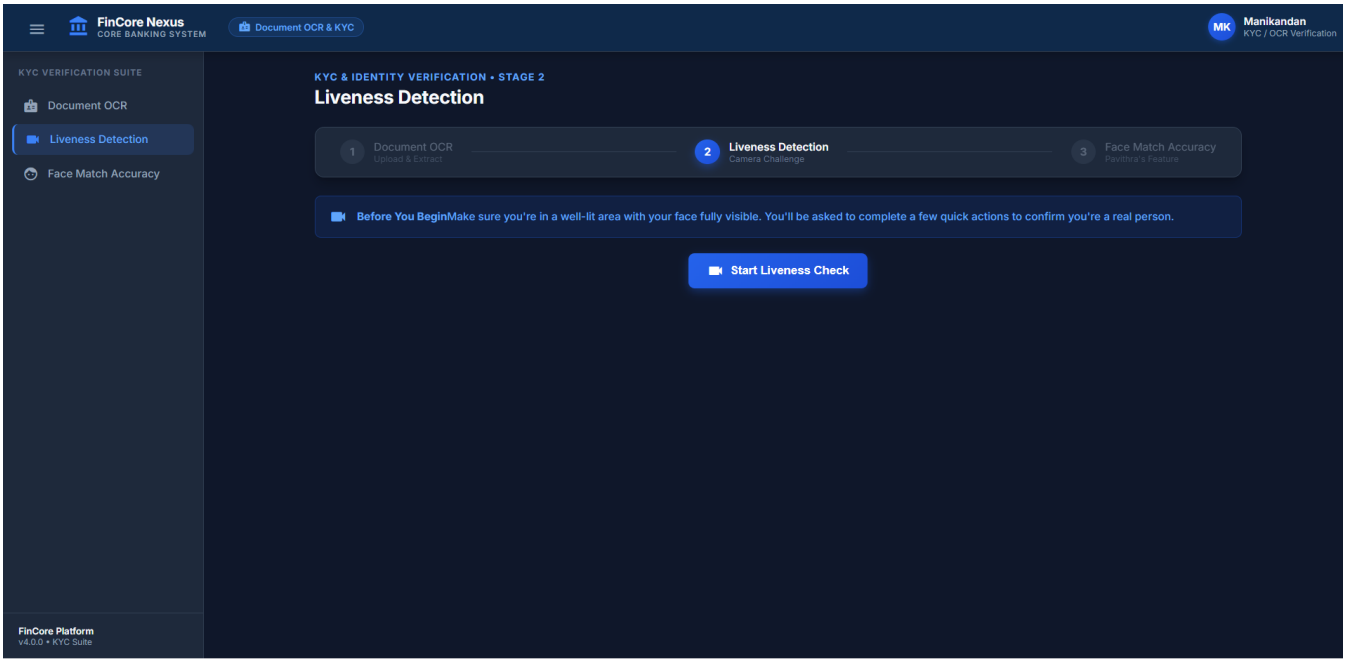

Figure 2 — Liveness Detection

7. Module 3 — Face Match Verification

The final AI check compares the customer's live selfie against their registered document photo. The Spring Boot backend exposes a multipart /api/face/verify endpoint that forwards both images to a Python FastAPI microservice, which runs DeepFace's VGG-Face model (OpenCV detector backend) to compute a verified/not-verified verdict, a distance score and the model's threshold.

Method	Endpoint	Service	Description
GET	/api/face/test	Spring Boot	Health check for the face-match controller
POST	/api/face/verify	Spring Boot	Accepts registeredImage + selfieImage, calls the AI service
POST	/face-match	FastAPI (port 8001)	Runs DeepFace verification and returns matched/distance/threshold

Temporary image files are written for processing and deleted immediately afterward, and the persisted face_match table intentionally stores only the essential outcome — match_result (SUCCESS/FAILED) and a confidence percentage — keeping the schema minimal per the UI requirement.

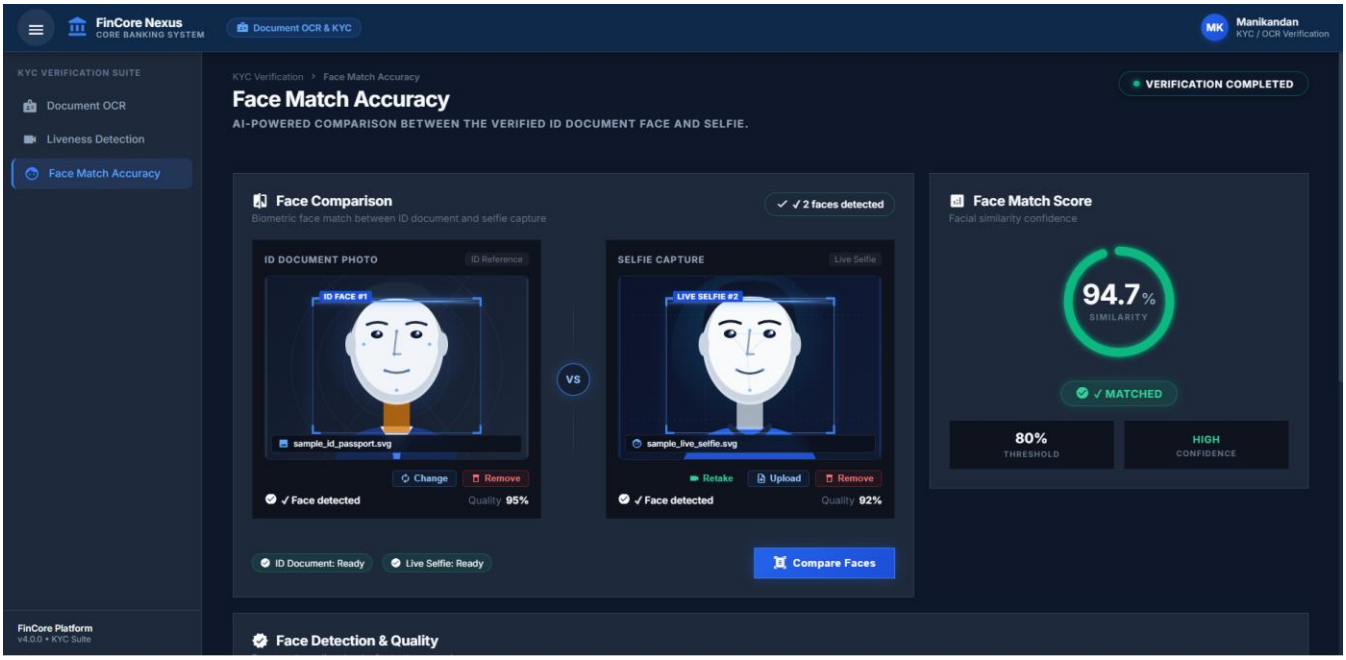

Figure 3 — Face Match

8. Database Design

The three new tables extend the existing digital_banking schema (MySQL, managed via XAMPP/phpMyAdmin) and relate back to the customer table already established in earlier milestones.

8.1 document_ocr

Field	Type	Key / Constraint	Description
ocr_id	BIGINT	PK, AUTO_INCREMENT	Unique OCR record ID
customer_id	BIGINT	FK	Links OCR result to customer
document_type	ENUM	NOT NULL	Aadhaar, PAN, Passport, Driving Licence, Voter ID, etc.
document_reference	VARCHAR(255)	—	Reference / path of uploaded document
extracted_name / id_number / dob / gender / address	VARCHAR/DATE/TEXT	—	Fields extracted by the AI OCR engine
ocr_confidence	DECIMAL(5,2)	0–100	OCR confidence score
ocr_status	ENUM	NOT NULL	Pending / Processing / Verified / Failed

Field	Type	Key / Constraint	Description
processed_at / created_at	TIMESTAMP	—/DEFAULT	Processing and record-creation timestamps

8.2 liveness_detection

Field	Type	Key / Constraint	Description
liveness_id	BIGINT	PK, AUTO_INCREMENT	Unique liveness record ID
customer_id	BIGINT	FK	Links result to customer
capture_reference	VARCHAR(255)	—	Reference / path of captured media
liveness_score	DECIMAL(5,2)	0–100	AI liveness confidence score
liveness_threshold	DECIMAL(5,2)	DEFAULT 80.00	Minimum passing threshold
detection_result	ENUM	NOT NULL	Pending / Passed / Failed
checked_at / created_at	TIMESTAMP	—/DEFAULT	Verification and record-creation timestamps

8.3 face_match

Kept intentionally minimal to match the M4 UI requirement:

Field	Type	Key / Constraint	Description
face_match_id	BIGINT	PK, AUTO_INCREMENT	Unique face match ID
customer_id	BIGINT	FK	Links result to customer
match_result	ENUM	NOT NULL	SUCCESS / FAILED
confidence	DECIMAL(5,2)	0–100	Face match confidence percentage
created_at	TIMESTAMP	DEFAULT	Record creation time

8.4 SQL & Integration Assets

- SQL scripts: 01_create_document_ocr.sql, 02_create_liveness_detection.sql, 03_create_face_match.sql, 04_m4_sample_data_and_queries.sql.
- PHP integration layer (db_connect.php, ocr_result.php, liveness_detection.php, face_match.php, m4_verification_result.php) acting as a lightweight API between the AI services and MySQL for the database team's own testing.

9. Conclusion

Milestone 4 successfully layers an AI-assisted KYC pipeline onto the Secure platform: customers can upload an ID document, prove liveness on camera, and have their selfie matched to their document photo, with every step's outcome durably recorded against their customer_id for compliance and audit. The three-table schema stays intentionally lean while the service layer (Spring Boot + FastAPI) keeps AI computation decoupled from the core banking backend.

10. Team Members

Team Member	Role
Manikandan	Frontend
Pavithra	Frontend
Kousalya	Frontend
Nithish	Backend
Jeevana	Backend
Raghvendra	Database